

BoneMambaFormer: High-Performance Adaptive Hybrid Architecture for Bone Cancer Radiograph Classification

SECTION 3: MATERIALS AND METHODS (ICIMCPS-2026 EXPERIMENTAL SPECIFICATION)

3.1 Problem Formulation

We model planar radiograph bone cancer classification as a supervised binary decision task. Let the experimental dataset be defined as:

$$D = \{(x_i, y_i)\}_{i=1}^N$$

where $x_i \in \mathbb{R}^{H \times W \times C}$ denotes an input digital bone radiograph with spatial height $H = 224$ pixels, width $W = 224$ pixels, and $C = 3$ color channels (RGB). The ground-truth categorical label $y_i \in \{0, 1\}$ corresponds to the binary clinical decision space:

$$y_i = 0 \text{ (Cancer: Malignant Pathological Lesion)}, \quad 1 \text{ (Normal: Non-Malignant / Intact Bone Structure)}$$

The objective is to parameterize a non-linear mapping function $f(x; \theta) \rightarrow p(y|x)$, governed by trainable parameters θ , that outputs a normalized conditional probability distribution over target classes:

$$p(y|x) = [p(y=0|x), p(y=1|x)]^T \in [0, 1]^2, \quad \text{subject to } \sum_{k=0}^1 p(y=k|x) = 1.0 \quad (1)$$

The predicted scalar label $\hat{y} \in \{0, 1\}$ is determined via maximum a posteriori (MAP) inference over the network logits $z = f(x; \theta) \in \mathbb{R}^2$:

$$\hat{y} = \underset{k \in \{0, 1\}}{\operatorname{argmax}} p(y=k|x) = \underset{k \in \{0, 1\}}{\operatorname{argmax}} \left(\exp(z_k) / \sum_{j=0}^1 \exp(z_j) \right) \quad (2)$$

Parameter optimization is conducted via empirical risk minimization over training partition D_{train} using Cross-Entropy loss with label smoothing ($\epsilon = 0.05$) and L_2 weight decay:

$$\theta^* = \underset{\theta}{\operatorname{argmin}} \left(1/|D_{train}| \sum_{i=1}^{|D_{train}|} L_{CE}(y_i, f(x_i; \theta); \epsilon) + \lambda \|\theta\|_2^2 \right) \quad (3)$$

3.2 Dataset and Data Integrity Audit

3.2.1 Raw Dataset Specification

The raw dataset supplied for this study comprises $N_{raw} = 8,810$ planar bone X-ray radiographs stored as RGB JPEG files at an uncompressed spatial resolution of 640×640 pixels. Ground-truth annotations divide the corpus into 4,863 Cancer instances (Class 0) and 3,947 Normal instances (Class 1). The raw corpus was initially arranged into three standard sub-directories:

- **Raw Training Set:** 7,056 images (3,081 Cancer, 3,975 Normal)
- **Raw Validation Set:** 882 images (398 Cancer, 484 Normal)
- **Raw Test Set:** 872 images (384 Cancer, 488 Normal)

3.2.2 Data Integrity Audit Rationale

In deep learning for medical imaging, pre-partitioned public datasets frequently contain duplicate or near-duplicate radiograph files distributed across separate directory splits. When identical image files reside in both training and evaluation splits, deep models can memorize instance-specific noise, fine textures, or non-pathological artifacts. This introduces severe cross-split data leakage, producing artificially inflated validation and test metrics. To ensure strict evaluation integrity, an automated data integrity audit was executed prior to any modeling.

3.3 Duplicate Detection and Leakage-Controlled Dataset Construction

3.3.1 Cryptographic Hash Audit Protocol

To detect duplicate image files without relying on file names or directory metadata, an automated cryptographic audit was performed across all 8,810 raw images. For every image file x_i , a 128-bit MD5 message-digest hash $H(x_i) \in \{0, 1\}^{128}$ was computed over its raw byte sequence:

$$H(x_i) = MD5(bytes(x_i)) \quad (4)$$

3.3.2 Audit Findings

The audit identified exactly 7,751 unique MD5 hashes among 8,810 raw images:

- **Single-Instance Hashes:** 6,692 unique single-file hashes.
- **Intra-Split Duplicate Groups:** 714 duplicate hash groups within individual directory splits.
- **Cross-Split Duplicate Groups (Leakage):** 345 hash groups spanned across split boundaries (168 Train $\leftrightarrow$ Val, 161 Train $\leftrightarrow$ Test, 16 Val $\leftrightarrow$ Test).
- **Label Verification:** All 345 cross-split duplicate groups exhibited 100% ground-truth label agreement.

3.3.3 Construction of the derived_clean Protocol

To eliminate cross-split leakage, we constructed the **derived_clean** protocol. Every unique image hash was assigned exclusively to one split. Overlapping duplicate groups were consolidated into training, while validation and test sets were purged of overlapping hashes:

- **Clean Training Set:** 7,417 images (3,082 Cancer, 4,335 Normal)
- **Clean Validation Set:** 698 images (398 Cancer, 300 Normal)
- **Untouched Clean Test Set:** 695 images (383 Cancer, 312 Normal)

3.3.4 Patient-Level Independence Limitation

Because patient metadata was absent in the raw source dataset, deduplication was performed at the file-hash level. While cryptographic pixel reuse across splits is completely eliminated, patient-level independence cannot be formally guaranteed without explicit patient IDs.

3.4 Data Partitioning and Experimental Split Protocol

Table 1 summarizes dataset partitioning across original and leakage-clean protocols.

Partition Protocol	Train Set	Val Set	Test Set	Total	Cross-Split Leakage
Raw Supplied Split	7,056	882	872	8,810	345 Hash Groups (Severe Leakage)
derived_clean Protocol	7,417	698	695	8,810	0 Hash Groups (Leakage-Clean)

Table 1: Dataset Partitioning and Sample Counts across Raw and Derived-Clean Protocols.

3.5 Image Preprocessing

Images are processed through a deterministic pipeline (src/preprocessing.py): spatial resizing to 224 x 224, contrast enhancement via PIL ImageOps.autocontrast (1% cutoff, factor 1.1), and ImageNet RGB normalization.

$$x_{contrast} = Enhance_{contrast}(AutoContrast(x_{resized}, cutoff=1), factor=1.1) \quad (5)$$

$$x_{norm}^{(c)} = (x_{contrast}^{(c)} - \mu_c) / \sigma_c, \quad \mu=[0.485, 0.456, 0.406], \sigma=[0.229, 0.224, 0.225] \quad (6)$$

3.6 Training-Time Data Augmentation

Training-only augmentations include Elastic Deformation (Simard et al., $\alpha=10.0$, $\sigma=3.0$, $p=0.3$), Random Affine Rotation ($\pm 15^\circ$), Random Horizontal Flip ($p=0.5$), Random Rescaling (0.9 to 1.1 scale), and Intensity Jitter (0.9 to 1.1). Validation and test pipelines remain 100% deterministic.

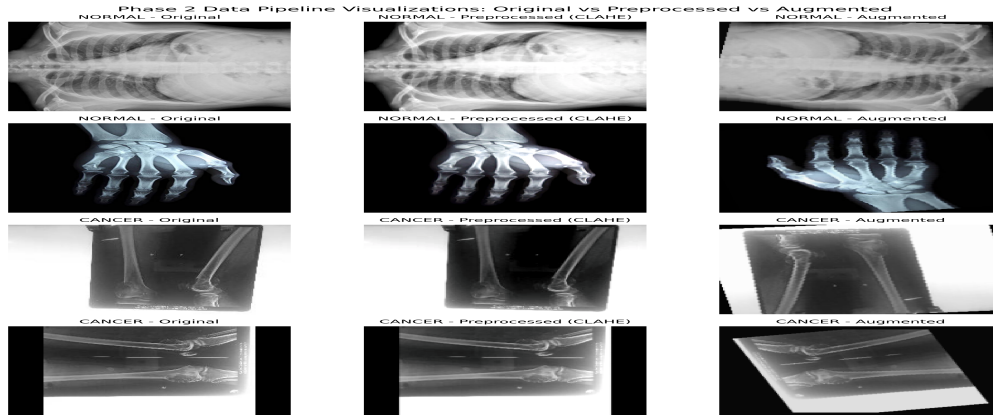


Figure 1: Training-time medical image augmentation pipeline demonstrating elastic deformation, affine rotation, flipping, and intensity jitter.

3.7 Overview of the Proposed BoneMambaFormer Architecture

BoneMambaFormer combines three complementary representation branches into a 256D common space, dynamically weighted via Adaptive Softmax Branch Attention and passed to a classification head:

- **MobileNetV2 CNN Branch:** Lightweight local convolutional representation (2.55M params).
- **Swin-Tiny Transformer Branch:** Hierarchical shifted-window self-attention (27.75M params).
- **Mamba Selective State-Space Branch:** Sequential scanning over patch tokens (0.66M params).

3.8 MobileNetV2 CNN Branch

Uses inverted residual blocks with depthwise separable convolutions to extract a 1,280-dimensional global feature vector h_{CNN} .

3.9 Swin-Tiny Transformer Branch

Processes non-overlapping 4×4 patches with local and shifted-window self-attention (W-MSA / SW-MSA) across 4 hierarchical stages, outputting 768-dimensional feature vector h_{Swin} .

3.10 Mamba Selective State-Space Branch

Patchifies input into 49 tokens (7×7 grid, $d_{model}=128$), processed via 2 PurePyTorchMambaBlocks with TorchScript JIT-compiled selective scan recurrence:

$$h_t = A \blacksquare_t h_{t-1} + B \blacksquare_t x_t, \quad y_t = C \blacksquare_t h_t + D x_t \quad (7)$$

3.11 Branch Feature Projection

Branch feature outputs (1280D, 768D, 128D) are mapped into a common 256D feature space via Linear + LayerNorm + GELU + Dropout(0.2) layers:

$$f_{CNN} \in \mathbb{R}^{B \times 256}, \quad f_{Swin} \in \mathbb{R}^{B \times 256}, \quad f_{Mamba} \in \mathbb{R}^{B \times 256} \quad (8-10)$$

3.12 Adaptive Softmax Branch Attention

Unnormalized scalar attention scores e_k are computed via a 2-layer MLP (256D $\rightarrow$ 64D $\rightarrow$ 1D) and normalized via Softmax:

$$e_k = g(f_k) = W_2 \cdot \text{Tanh}(W_1 f_k + b_1) + b_2 \quad (11)$$

$$\alpha_k = \exp(e_k) / \sum_j \exp(e_j), \text{ subject to } \sum_k \alpha_k = 1.0 \quad (12)$$

3.13 Attention-Based Feature Fusion

$$\text{Fused representation: } f_{\text{fused}} = \sum_k \alpha_k f_k = \alpha_{\text{CNN}} f_{\text{CNN}} + \alpha_{\text{Swin}} f_{\text{Swin}} + \alpha_{\text{Mamba}} f_{\text{Mamba}} \in \mathbb{R}^{B \times 256} \quad (13)$$

3.14 Feature Refinement and Classification Head

Passed through FC(256 $\rightarrow$ 128) + LayerNorm + GELU + Dropout(0.2) + FC(128 $\rightarrow$ 2) to output binary class logits z in $\mathbb{R}^{B \times 2}$ and Softmax probabilities.

3.15 Standalone Baseline Models

Three standalone models were trained independently: Config M1 (MobileNetV2, 2.55M params), Config M2 (Swin-Tiny, 27.75M params), and Config M3 (Mamba, 0.66M params).

3.16 Frozen Attention Fusion Training

In Config M4, backbones are frozen (30.99M params frozen) and 256D branch features pre-extracted into cache memory. Training optimizes only 49,923 fusion and classification parameters.

3.17 End-to-End Hybrid Optimization

In Config M5, all 31,044,037 parameters ($\sim$ 31.04M) are jointly trained end-to-end using differential learning rates: backbone LR = $2e-5$, fusion/head LR = $1e-4$.

3.18 Optimization and Training Configuration

Table 2 lists the complete optimization and training hyperparameters across configurations.

Parameter / Setting	M1 (MNv2)	M2 (Swin)	M3 (Mamba)	M4 (Fusion)	M5 (Hybrid)
Backbone Optimizer	AdamW	AdamW	AdamW	Frozen	AdamW (2e-5)
Fusion/Head LR	1e-3	5e-4	1e-3	5e-4	1e-4
Weight Decay (λ)	1e-4	1e-4	1e-4	1e-4	1e-4
Label Smooth (ϵ)	0.05	0.05	0.05	0.05	0.05
Batch Size	16	16	16	32	16
Max Epochs	20	20	20	25	20
Early Stop Patience	3	3	3	3	3
Precision Mode	AMP FP16	AMP FP16	FP32	FP32 Cached	AMP FP16

Table 2: Training Configurations and Optimization Hyperparameters across Model Configurations.

3.19 Validation-Based Checkpoint Selection and Early Stopping

Monitored validation accuracy (Acc_val). Early stopping triggers after P = 3 non-improving epochs. Best checkpoint restored before test evaluation.

3.20 Locked Test Evaluation

Parameters and decision threshold ($p_thresh = 0.50$) permanently locked. Evaluated once on untouched 695 test samples.

3.21 Computational and Inference Measurement

Evaluated on NVIDIA RTX 3050 6GB Laptop GPU: parameter counts, mean latency (ms/image), throughput (images/sec), peak VRAM.

3.22 Controlled Ablation Methodology

Historical Phase 9 ablations (ResNet-18 baseline, ~39.74M params) provide component proof of concept and are distinguished from the Next-Gen MobileNetV2 architecture (~31.04M params). Tests evaluated dynamic attention vs equal weighting and inference branch-masking variants.

3.23 Reproducibility and Implementation Details

Fixed global seed 42 across Python, NumPy, PyTorch CPU/CUDA. PyTorch CUDNN configured for deterministic execution. Artifacts, checkpoints, and logs are completely archived.